

Product and Design Specification

A reading and publishing platform built on the SceneCraft pipeline

Four roles: Reader, Writer, Publisher, Admin

Page-by-page reader | Writer templates | Publisher requests and chat

Wattpad-inspired layout and colours, original logo and content

Images on Cloudinary | Copyright and takedown flow

Version 1.0 | 3 October 2026 | Prepared for a local rebuild inside the existing SceneCraft folder

Design references researched from wattpad.com on 3 October 2026 (see Sources).

Contents

Contents	2
1. Summary and decisions so far	3
2. Roles and permissions (RBAC)	4
3. User journeys	5
4. The page-by-page reader	6
5. Writer templates	7
6. Publisher requests and chat	9
7. Dashboards	10
8. Images with Cloudinary	11
9. Data model	12
10. API routes	13
11. Frontend pages by role	14
12. Design: Wattpad-inspired look and feel	15
13. Copyright and moderation	20
14. Rebuild plan on the existing SceneCraft folder	20
15. Open questions	22
Sources	22

1. Summary and decisions so far

The goal is to turn SceneCraft from a single-user analysis tool into a Wattpad-style platform. Writers publish a book as one manuscript document with a cover image. Readers browse a catalogue, read page by page and use SceneCraft's analysis features on the book they are reading. Publishers read the book, see a condensed view of the analysis and send the writer a publishing request. Admins watch platform health. The existing 10-stage BullMQ pipeline, workers, models, graph and charts stay; a product layer is added on top.

Topic	Decision
Roles	Four roles with RBAC: admin, writer, reader, publisher. Admin is seeded, never self-signup. Publishers apply and are approved by an admin before they can send requests.
Publishing format	Writer uploads one manuscript (PDF, DOCX or TXT) plus a cover image. No chapter-by-chapter publishing in v1.
Reading experience	Single document read page by page, like Google Play Books: page turns, scrubber, resume where you stopped, font and theme settings.
SceneCraft features	Readers get the analysis tabs on every book. Publishers get a lighter, read-only pitch view. Continuity Checker stays writer-only.
Writer templates	Writer chooses one of 3 layouts for how their book page looks to readers (Classic, Showcase, Notebook) plus an accent colour.
Publisher flow	Browse and read, star a book to a wishlist, click "I want to publish" and fill a details form. The writer gets a request, accepts or declines, and chat opens on acceptance.
Login	Email and password only for now. A Google sign-in slot is reserved in the UI.
Image storage	Cloudinary for covers and avatars. MongoDB stores only the publicId and URL. Manuscript files stay private (section 8).
Look and feel	Layout, flow and colour family follow Wattpad (orange primary, white surfaces, cover-led grids). Logo is yours. Copy, illustrations and covers are original (section 12).
Copyright	Rights checkbox on publish, Report button, admin queue, strikes and takedown page (section 13).

Scope note. This document is a build brief, not legal advice. Have a lawyer review the Terms, Privacy Policy and takedown text before the platform goes public.

2. Roles and permissions (RBAC)

Add **role** (admin, writer, reader, publisher) and **status** (active, pending, banned) to the User model. After the existing JWT guard, add an **authorize(...roles)** middleware. Put the role in the JWT payload and re-check ownership in controllers (a writer may only edit their own books).

Role	How they get it	Can do	Cannot do
Reader	Self sign-up (default).	Browse and search catalogue; read any published book; use analysis tabs on what they have read; rate and review; save to reading list; follow writers; report content.	Publish books; see Continuity Checker; message publishers or writers.
Writer	Self sign-up (choose Writer) or upgrade from reader in settings.	Upload and publish books with cover; choose page template; edit blurb, tags and genre; see analytics dashboard; read reviews; accept or decline publisher requests; chat with publishers who were accepted; see Continuity Checker on own books; unpublish.	See other writers' private data; edit other books; message publishers first.
Publisher	Applies at sign-up with company name and website. Admin approves.	Browse and read like a reader; see the pitch view (arc, mood, cast, traction); star books to a wishlist; send "I want to publish" requests; chat once a request is accepted.	Publish books; see Continuity Checker; leave public reviews (keeps ratings honest); send requests before approval.
Admin	Seeded account (script or env). Never from the sign-up form.	Platform dashboard; approve or reject publishers; ban or change roles; review reports; unpublish books; issue copyright strikes.	Read private chats unless investigating a report (log every such access).

Enforcement checklist

- Every route declares its allowed roles in one place (route file) so the matrix above stays auditable.
- Frontend uses a RoleRoute component only for UX. The server is the real gate.
- Pending publishers can log in but see a "waiting for approval" screen and cannot send requests.
- Banned users get 403 on all write routes and cannot log in again; their books are hidden, not deleted.
- Rate limit login, sign-up, review posting, reports and chat messages (Redis, already in your stack).

SceneCraft feature access

Not every analysis feature should be public. Readers see structure that helps reading; publishers see a summary that helps judging; the Continuity Checker exposes the writer's mistakes so it stays private.

Feature	Writer (own book)	Reader	Publisher
Scene breakdown (cards, locations, cast)	Full	Read-only, spoiler filtered	Hidden
Character profiles	Full, editable merges	Read-only, spoiler filtered	Read-only (main cast)
Relationship graph	Full	Read-only, spoiler filtered	Read-only
Story timeline (narrative / chronological)	Full	Read-only, spoiler filtered	Hidden
Mood and emotion analysis	Full	Read-only, spoiler filtered	Whole-book summary
Story arc and climax chart	Full	Read-only, spoiler filtered	Full (spoilers allowed)
Semantic search	Full	Full, results limited to pages read	Hidden
Continuity Checker	Full (triage dashboard)	Hidden	Hidden
Pitch card (logline, genre, tone, audience, comparable titles)	View and regenerate	Hidden	Full
Reader traction (reads, rating, completion, wishlists)	Full	Reads and rating only	Full summary

Spoiler protection. Store the reader's furthest page. Filter characters, graph edges, scenes and timeline events to those whose source offset falls on or before that page. A "show everything" toggle lets readers opt out. Because scenes already store text offsets, the filter is a simple offset comparison.

3. User journeys

Writer

- 1 Signs up as Writer, lands on the writer dashboard.
- 2 Clicks Publish new: uploads manuscript (PDF, DOCX or TXT), uploads a cover (cropped to 2:3), enters title, blurb, genre, tags, language and maturity.
- 3 Chooses a page template (Classic, Showcase, Notebook) with live preview and an accent colour.
- 4 Ticks the rights checkbox ("I own this work or have the rights to publish it") and submits.
- 5 Sees live pipeline progress over Socket.io (existing feature). Book stays Draft until parsing and pagination finish, then can be set Published.
- 6 Reads dashboard: reads, profile views, rating, reviews, drop-off by page, reading-list adds, publisher wishlist count.
- 7 Receives a publisher request, reads the details, accepts or declines with a note. On accept, chat opens.

Reader

- 1 Lands on the public landing page, signs up or logs in, arrives at the Home feed (Trending, genres, recommended).
- 2 Opens a book page, reads the summary, sees stats and reviews, taps Start reading or Add to reading list.
- 3 Reads page by page. Resumes automatically next time. Opens the Insights drawer for characters, graph, timeline, mood, arc and search.
- 4 Rates and reviews the book, and can report it. Manages status in My Library: Reading, Want to read, Finished.

Publisher

- 1 Applies as Publisher (company, website, short note). Waits for admin approval.
- 2 Browses a Discover page that looks like the reader catalogue, with extra filters: rating, completion, genre, length, wishlists.
- 3 Opens a book: can read it fully and open the Pitch panel (arc, mood summary, cast, traction, writer snapshot, pitch card).
- 4 Stars books to a Wishlist (private to them).
- 5 Clicks **I want to publish**, fills the request form, sends. Status becomes Pending.
- 6 After the writer accepts, a chat thread opens in the Requests tab. Either side can close it. Publisher can withdraw a pending request.

Admin

- 1 Logs in with the seeded account and sees the platform dashboard.
- 2 Approves or rejects publisher applications, manages users, handles reports and takedowns.

4. The page-by-page reader

Readers should feel like they are turning pages in one continuous document, similar to Google Play Books, instead of jumping between chapters. SceneCraft already parses the manuscript to text and stores scene offsets, so the cleanest approach is to paginate that text once on the server and reuse the offsets everywhere.

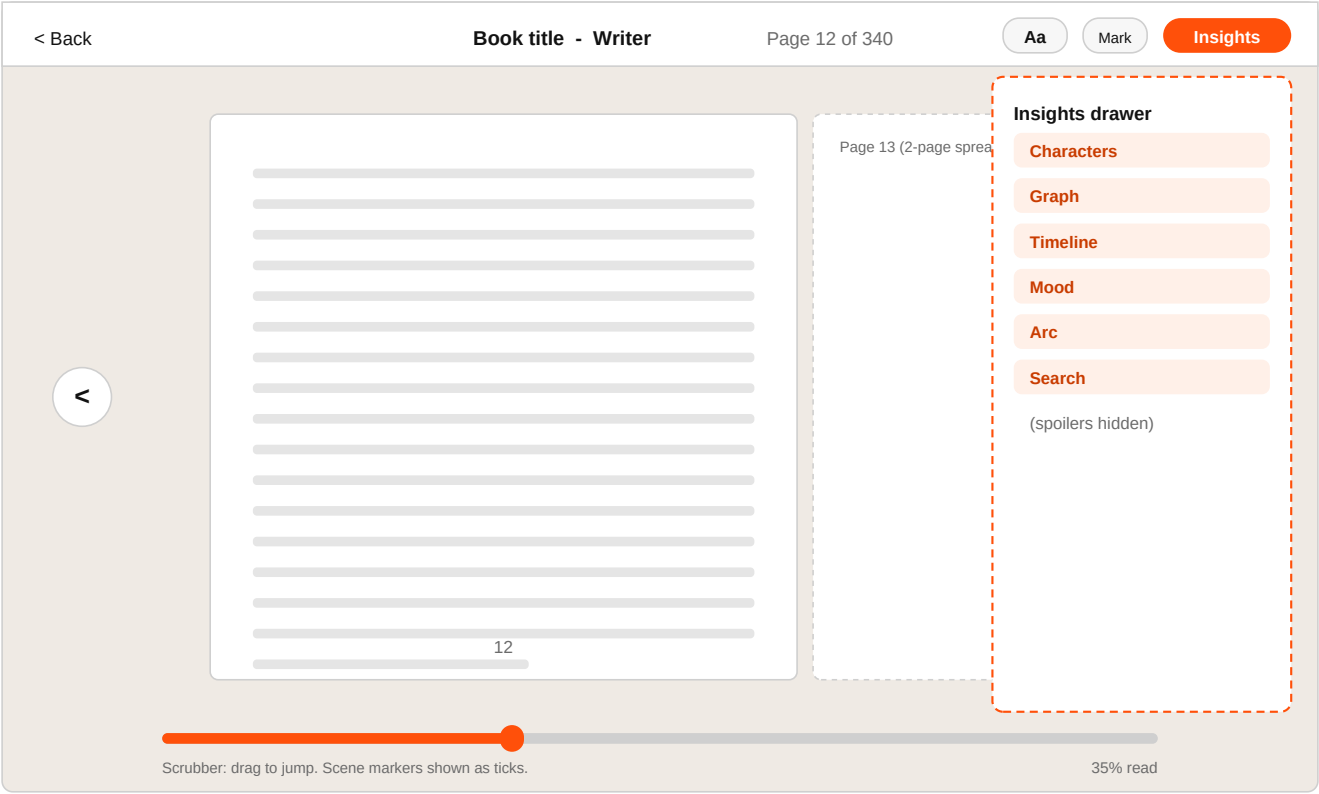
How pagination works

- 1 On upload, store the original file in GridFS (so the writer can re-download it) and parse it with the existing pdf-parse, mammoth or plain-text parser.
- 2 A new **paginator service** splits the clean text into pages of roughly 1,800 characters, breaking only at paragraph or sentence boundaries. Store **pageOffsets[]** (start index of each page) on the Book.
- 3 To find which page a scene or character mention is on, binary-search pageOffsets with its stored offset. This powers spoiler filtering, scene ticks on the scrubber and "jump to scene".
- 4 Pages are served in small windows (for example pages 10 to 14) so the client never downloads the whole book at once.

Why not render the original PDF? A PDF viewer keeps the original layout but cannot reflow for phones, cannot change font size and makes offset-to-page mapping harder. Text pages are consistent with your analysis offsets. If a writer needs exact layout (poetry, illustrated books) you can add an optional "original PDF mode" with pdf.js later.

Reader screen behaviour

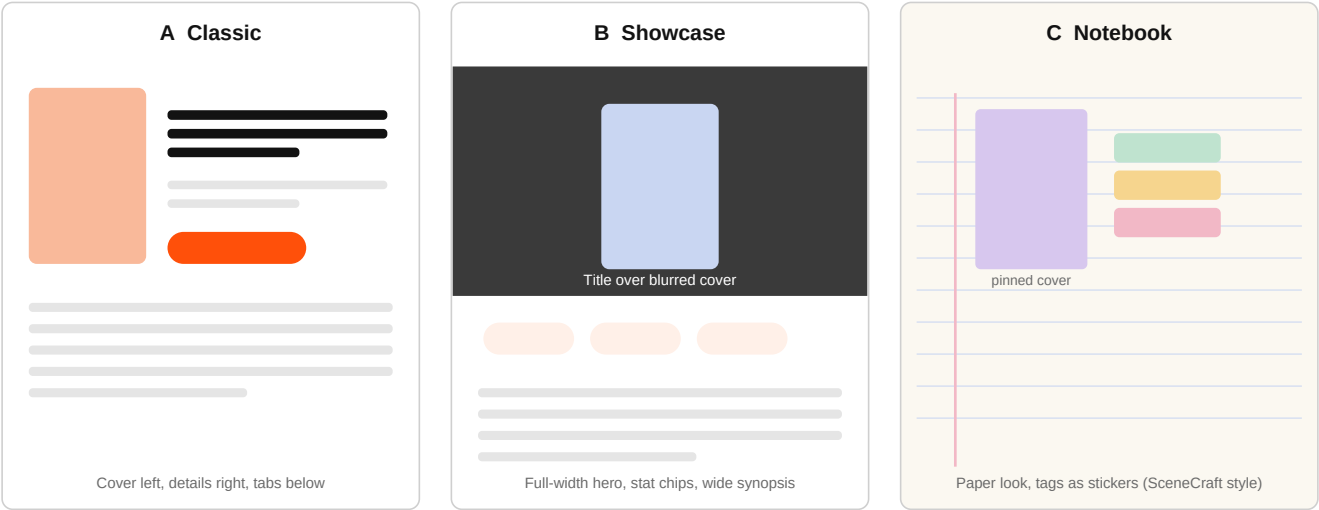
Element	Behaviour
Page turn	Arrow buttons, keyboard arrows, click on left or right edge, swipe on touch. Short slide or flip animation (Framer Motion, already installed).
Page counter and scrubber	"Page 12 of 340" and a draggable slider along the bottom. Scene boundaries appear as small ticks. Jump-to-page input.
Resume	Save the current page after each turn (debounced). Reopening a book jumps to it. Also store furthest page for spoilers.
Reader settings	Font size, line spacing, serif or sans, theme: light, sepia, dark. Saved per user.
Bookmarks	Tap Mark to bookmark a page; list in the drawer.
Wide screens	Optional two-page spread. Mobile always shows one page.
Insights drawer	Slides in from the right with tabs: Characters, Graph, Timeline, Mood, Arc, Search. Everything is filtered to pages read unless the toggle is on.
Finish	On the last page show a completion card: rate this book, write a review, add to Finished, similar books.



Wireframe: reader screen with page view, scrubber and insights drawer.

5. Writer templates

Each book gets a public book page. The writer picks one of three layouts and an accent colour. All three templates show the same data (cover, title, author, stats, summary, tags, Insights tab, Reviews tab, Start reading button). Only the layout and mood change, so you build one data component and three layout wrappers.



Wireframe: the three writer templates.

Template	Layout	Best for	Notes
A. Classic	Cover on the left, title, author, stats and buttons on the right, tabs below. Closest to the familiar Wattpad story page.	Most books; default.	Fastest to build. Use as the base.
B. Showcase	Wide hero with blurred, darkened cover as backdrop, centred cover, stat chips, wide synopsis, author card to the side.	Cover-led genres: fantasy, thriller, romance.	Generate backdrop with CSS blur on the cover image; no extra upload.

Template	Layout	Best for	Notes
C. Notebook	Paper background and ruled lines, cover "pinned" on the page, tags as sticker chips, handwritten-style headings. Echoes SceneCraft's original notebook look.	Literary fiction, poetry, memoir, essays.	Uses a small handwritten font for headings only; keep body text clean.

How it is stored and applied

- Book fields: **template** (classic, showcase, notebook) and **accent** (one of six preset hex colours to keep contrast accessible).
- The writer sees a live preview in the Publish and Edit screens and can change it any time without re-running analysis.
- Optionally set a default template on the writer profile so new books start with it.
- The same template wrapper can later style the writer's profile page (stats, bio, book shelf).

6. Publisher requests and chat

The pitch view

Publishers open the same reader, plus a Pitch panel on the book page. It shows what helps a publisher decide quickly:

- Story arc chart with climax markers and a pacing summary.
- Whole-book mood summary (dominant emotions, intensity range).
- Main cast list with roles and a read-only relationship graph.
- Reader traction: reads, rating, review count, completion rate, reading-list adds, number of publishers who wishlisted.
- Writer snapshot: other books, followers, how often they publish.
- AI pitch card (one extra LLM call, cached): logline, genre, tone, target audience, "for fans of".

"I want to publish" request form

Opens a modal from the book page. All fields below are filled by the publisher and sent to the writer.

Field	Type	Notes
Company / imprint name	Text, required	Prefilled from the approved publisher profile.
Contact name, email, phone (optional)	Text, required	Prefilled; shown to the writer only after sending.
Website	URL	Prefilled.
Rights requested	Multi-select	Print, e-book, audiobook, translation, film/TV/web series.
Territory and language	Text or select	For example: India, English; worldwide.
Proposed terms	Text, required	Advance, royalty percentage or flat fee. Free text in v1.
Timeline	Select or text	For example: decision in 2 weeks, release in 6 months.
Message to the writer	Long text, required	Why this book fits the publisher.

Request lifecycle

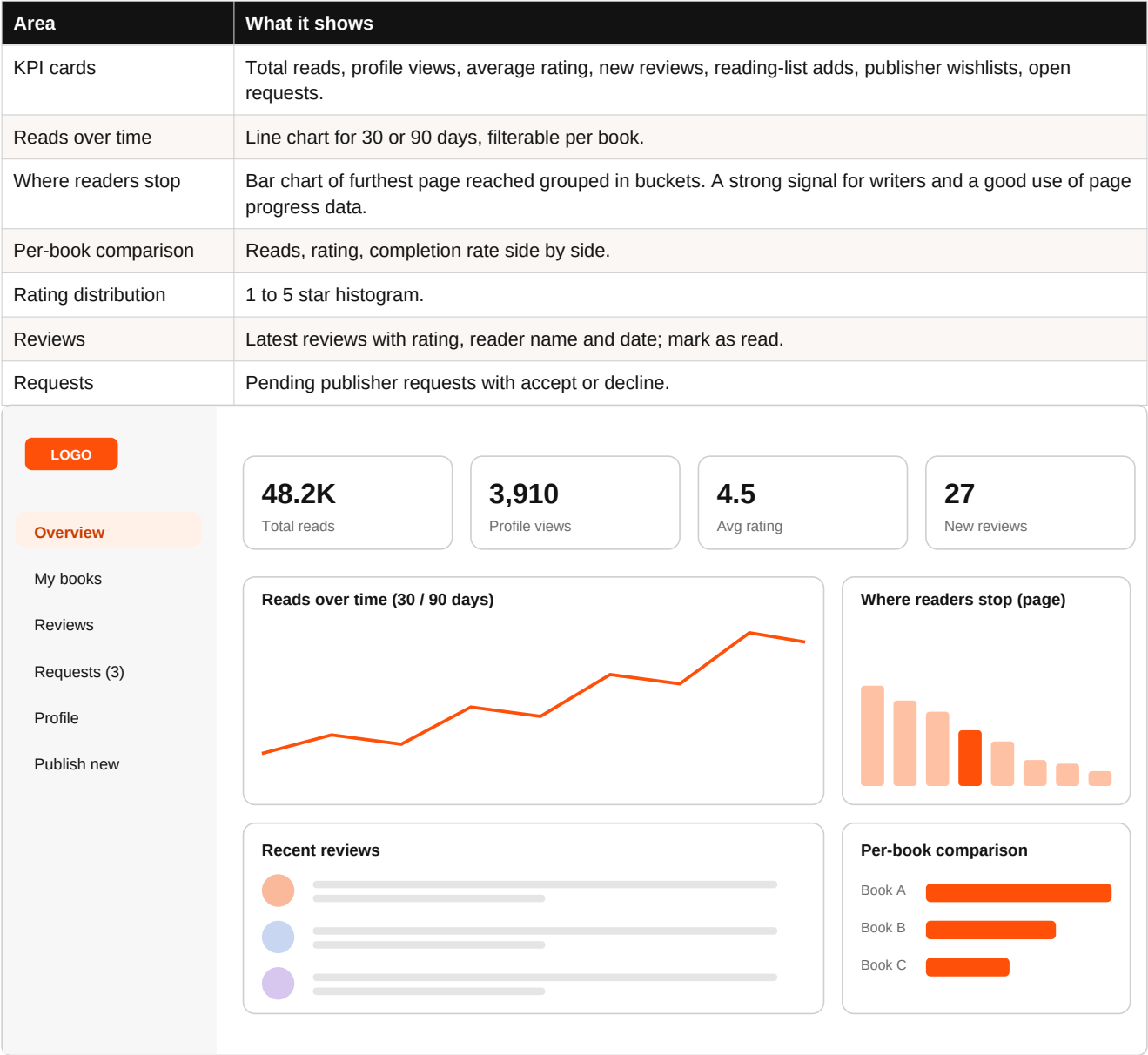
State	Who moves it	Result
Pending	Publisher sends	Writer gets a notification and an inbox item. Publisher cannot send a second request for the same book while one is active.
Accepted	Writer	A Conversation is created between both users. Both can chat in real time (Socket.io). Publisher's wishlist star turns to "In talks".
Declined	Writer (optional note)	Publisher is notified. A new request is allowed after a cooldown, for example 30 days.
Withdrawn	Publisher	Only while Pending. Closes silently on the writer's side.
Closed	Either side	Ends the chat; history remains readable to both.

Why chat opens only after acceptance. Writers should not get cold messages from anyone who calls themselves a publisher. The request form is the single door in, and the admin approval keeps fake publishers out. If you want open chat from the start, tell me and I will change it.

7. Dashboards

Writer dashboard

Shows how the writer's published work is doing and what readers think. Charts use Recharts, already in your stack.



Wireframe: writer dashboard.

Tracking events

Write small ViewEvent documents (type, targetId, viewerId or anonymous id, date). A nightly job rolls them into a DailyStat collection so dashboards read pre-aggregated numbers. Count a book view once per viewer per day, and a profile view once per viewer per day, to avoid inflated numbers.

Admin dashboard (simple)

- Users by role, new sign-ups over time, daily and weekly active users.
- Books published, total reads, reviews posted, average rating.
- Pending publisher approvals and open reports with quick action buttons.
- Top books and top writers this week.
- Users table with search, role change, ban and unban.

8. Images with Cloudinary

Cloudinary is a good fit for covers and avatars. It stores the image, resizes and compresses it through the delivery URL, and serves it from a CDN. That replaces the resize step, the image-serving route and the cache headers that a MongoDB-based approach would need.

Piece	Approach
What MongoDB stores	Only publicId and url (the <code>secure_url</code> Cloudinary returns) on the Book (cover) and User (avatar). Never the image bytes.
Upload path (recommended for v1)	Browser sends the file to your API with <code>multer</code> (memory storage). The API checks role, type and size, then calls <code>cloudinary.uploader.upload_stream</code> and saves the <code>publicId</code> and <code>url</code> . Simple, and nothing reaches Cloudinary without passing your checks.
Later option	Signed direct upload from the browser: your API issues a short-lived signature and the browser uploads straight to Cloudinary. Saves server bandwidth; add it only if uploads become heavy.
Folders	<code>platform/covers</code> and <code>platform/avatars</code> , so you can find and clean up assets in the Cloudinary console.
Sizes through the URL	Cover on the book page: <code>c_fill,w_600,h_900,f_auto,q_auto</code> . Grid thumbnail: <code>c_fill,w_200,h_300,f_auto,q_auto</code> . Avatar: <code>c_fill,w_256,h_256,g_face,f_auto,q_auto</code> . Build URLs from the <code>publicId</code> in one helper (for example <code>coverUrl(publicId, "thumb")</code>) so sizes can change later without touching stored data.
Replacing and deleting	Call <code>cloudinary.uploader.destroy(publicId)</code> when a cover is replaced or a book is deleted, so unused images do not pile up.
Validation	Accept JPEG, PNG and WebP only. Cap the file at about 5 MB in <code>multer</code> , before anything is sent to Cloudinary.
Config	Install <code>cloudinary</code> and <code>multer</code> in backend. Add <code>CLOUDINARY_CLOUD_NAME</code> , <code>CLOUDINARY_API_KEY</code> and <code>CLOUDINARY_API_SECRET</code> to <code>backend/.env</code> . The API secret never goes to the frontend.

Keep manuscripts off public Cloudinary URLs. Cloudinary delivery URLs are public by default. Covers are meant to be public, but the uploaded PDF, DOCX or TXT is the writer's work, sometimes unpublished. Keep the original file in MongoDB GridFS (or behind private, authenticated delivery) and store the parsed text and page offsets in MongoDB. Only covers and avatars go to Cloudinary.

Check the plan limits. Cloudinary's free plan caps monthly storage, transformations and bandwidth, and limits file sizes. I have not verified the current numbers, so check Cloudinary's pricing page before launch. Using `f_auto` and `q_auto` and small grid thumbnails keeps usage low.

Keep it behind one service

Put all Cloudinary calls in **storage.service.js** with three functions: `saveImage`, `deleteImage` and `imageUrl`. Controllers never call Cloudinary directly, so moving to another provider later means changing one file.

Cover upload checklist

- Frontend: image picker with a 2:3 crop box and a live preview inside the chosen template.
- Backend: `multer` memory storage, validate type and size, upload to `platform/covers`, return `publicId` and `url`, save them on the Book.
- If a writer skips the cover, show a generated placeholder (title on a tinted block, drawn in the frontend) while `coverPublicId` is empty.

9. Data model

New collections sit alongside your existing ones. Scenes, Characters, Relationships and the other analysis models keep pointing at the document and gain a bookId reference, so the pipeline needs only small changes.

Collection	Key fields	Notes
User (extend)	name, email, passwordHash, role, status, avatarPublicId, avatarUrl, bio, publisherProfile {company, website, note, approvedAt}, strikes	Role and status drive RBAC.
Book	writerId, documentId, title, blurb, coverPublicId, coverUrl, genre, tags[], language, mature, status (draft, processing, published, unpublished, removed), template, accent, pageCount, pageOffsets[], pitchCard, stats {reads, ratingAvg, ratingCount}, acceptedTermsAt, termsVersion	One book per manuscript Document. Cached stats keep lists fast.
Document (existing)	ownerId, fileId (GridFS), text, pipeline status	Add bookId. Analysis models stay linked to it.
Review	bookId, readerId, rating 1-5, text, createdAt, status	One review per reader per book. Recompute Book stats on write.
ReadingList	readerId, bookId, status (reading, want, finished), currentPage, furthestPage, bookmarks[], updatedAt	Doubles as reading progress. Unique on readerId+bookId.
Wishlist	publisherId, bookId, createdAt	Private to the publisher.
PublishRequest	bookId, writerId, publisherId, company, contact, rights[], territory, proposedTerms, timeline, message, status, writerNote, createdAt	One active request per publisher per book.
Conversation	requestId, participants[2], status, lastMessageAt	Created when a request is accepted.
Message	conversationId, senderId, text, readAt, createdAt	Delivered over Socket.io, persisted in Mongo.
Report	reporterId, targetType (book, review, user), targetId, reason, details, status, handledBy, outcome	Reasons: copyright, plagiarism, abuse, spam, other.
ViewEvent / DailyStat	type, targetId, viewerKey, date / aggregated counts per day	Rollup job keeps dashboards fast.

10. API routes

Route	Roles	Purpose
POST /auth/signup, /auth/login, GET /auth/me	public	Sign up as reader, writer or publisher applicant; JWT returned. Admin is never created here.
GET /books, GET /books/:id	public	Catalogue with filters (genre, tag, rating, length, status) and book detail.
POST /books	writer	Create a book: manuscript, cover, metadata, template, rights checkbox. Queues the pipeline.
PATCH /books/:id, DELETE /books/:id	owner, admin	Edit metadata and template; unpublish or delete.
GET /books/:id/pages?from=&to;=	reader, publisher, owner	Windowed page text for the reader.
GET /books/:id/analysis/:feature	by matrix in section 2	Existing analysis endpoints, role-gated and spoiler-filtered with ?upto=page.
GET /books/:id/pitch	publisher, owner	Pitch card and traction summary.
GET/POST /books/:id/reviews	reader	List and create reviews; PATCH and DELETE own review.
GET /me/library, PUT /me/library/:bookId	reader	Reading list, status and progress.
GET/PUT/DELETE /me/wishlist/:bookId	publisher	Star or unstar a book.
POST /publish-requests	publisher	Send "I want to publish".
GET /publish-requests, PATCH /publish-requests/:id	writer, publisher	List; writer accepts or declines; publisher withdraws.
GET /conversations, GET/POST /conversations/:id/messages	participants	Chat history and send; live updates through Socket.io.
GET /writer/analytics	writer	Dashboard numbers and chart series.
POST /reports	any logged-in	Report a book, review or user.
GET /admin/stats, /admin/users, /admin/reports	admin	Dashboard, users table, reports queue.
PATCH /admin/publishers/:id, /admin/users/:id, /admin/books/:id	admin	Approve publisher, change role or ban, take a book down, add strike.
POST /uploads/avatar	any logged-in	Upload a profile picture to Cloudinary; returns publicId and url. Covers are uploaded through POST /books and PATCH /books/:id.

11. Frontend pages by role

Area	Routes and pages
Public	/ Landing (guest) or Home feed (logged in) /login /signup /browse and /browse/:genre /search /book/:id (book page in the writer's template) /writer/:username (public profile) /terms /privacy /copyright
Reader	/read/:bookId (page-by-page reader with insights drawer) /library (Reading, Want to read, Finished) /settings
Writer	/w/dashboard /w/books /w/books/new (publish flow) /w/books/:id/edit (metadata, template, cover) /w/books/:id/insights (including Continuity Checker) /w/reviews /w/requests (inbox and chat)
Publisher	/p/discover /p/wishlist /p/requests (list and chat) /p/book/:id (pitch view with reading) /p/apply status
Admin	/a/overview /a/users /a/publishers /a/reports /a/books

After login, redirect by role: reader to /, writer to /w/dashboard, publisher to /p/discover, admin to /a/overview. Wrap each area in a RoleRoute. Keep one shared layout per area (top nav for public and reader, sidebar for writer, publisher and admin).

Proposed additions to your folders

```

backend/src/
  models/      Book Review ReadingList Wishlist PublishRequest Conversation Message Report
               ViewEvent DailyStat (User and Document extended)
  services/    storage.service paginator.service pitch.service stats.service spoiler.service
  middlewares/ authorize uploadCover rateLimit (extend)
  controllers/ book review library wishlist publishRequest chat report writerStats admin
  routes/      one router per controller, each declaring its roles
  workers/     (existing) + pitchWorker, statsRollupWorker
frontend/src/
  pages/       public/ reader/ writer/ publisher/ admin/
  components/  templates/ (ClassicTemplate ShowcaseTemplate NotebookTemplate)
               reader/ (PageView Scrubber ReaderSettings InsightsDrawer)
               common/ (BookCard CoverImage RoleRoute ReportButton StarButton)
  layouts/     PublicLayout WriterLayout PublisherLayout AdminLayout

```

12. Design: Wattpad-inspired look and feel

You asked for the UI and colours to be quite close to Wattpad. This section records how Wattpad is structured today (checked on 3 October 2026) and exactly how to translate it. The aim is the same familiarity and flow, with your own logo, wording, illustrations and covers.

12.1 What Wattpad does

Page	Observed structure
Landing (guest)	Top bar with logo, Browse, Community and Write dropdowns, Download app, Log in, Sign Up. A large hero headline with a subline, a Get started button, an "Already have an account? Log in" line, store badges and a hero image with floating comment and profile cards. Then, in order: Trending now (horizontal cover carousel with arrows), a genre section (illustrated tiles for Romance, Fanfiction, LGBTQ+, Werewolf, Fantasy, Teen Fiction, Historical Fiction, Paranormal, Humor, Horror, Mystery, Thriller, Science Fiction, Adventure, Non-Fiction, Poetry and more), a community or "book club" block with floating comments, a creators block with three author quotes and covers, an adaptations carousel, an app download block with QR code, and a wavy footer with socials and link rows.
Login	Orange logo top left, Sign up link top right, language selector. A device mock-up on one side. On the other, a card titled "Log in", buttons for Google and Facebook, an "or" divider, Log in with email, Forgot password and Sign up links. A "Get the app" phone-number box below. Same footer.
Story page	Same top bar. Cover, title, author avatar and username, a stats row (Reads, Votes, Parts), a Start reading button and a round add-to-library button. Tabs for Summary and Parts. A meta line (maturity, completion status, date, reading time), tag chips, a description with Read more, a sign-up prompt for guests, a "You may also like" carousel, and Report this story.
Footer links	Writers, Brand Partnerships, Jobs, Press, Terms, Privacy, Payment Policy, Accessibility, Help, copyright line.

12.2 What we keep, change and drop

Wattpad element	Our version
Browse / Community / Write dropdowns	Browse (genres, trending, new, complete), Write (publish a book, writer guide), For publishers.
Reads, Votes, Parts	Reads, Rating (stars from reviews), Pages.
Add to library button	Add to reading list (round + button, same position).
Summary and Parts tabs	Summary, Insights (SceneCraft features), Reviews. No chapter list, since reading is page by page.
Google and Facebook login	Email and password only for now; keep a visible slot for Google later.
Community and book-club block	Replace with a "Read deeper" block that shows the relationship graph and story arc. This is your differentiator.
Creators block with quotes	Replace with a "For writers" and "For publishers" pair of cards. Add real quotes only once you have them.
Adaptations carousel, QR app block, Premium, coins	Drop for v1.
Store badges and app promotion	Drop.

12.3 Colour, type and components

Check the orange. Wattpad's brand orange is very close to #FF500A in public brand-colour listings, but I could not confirm the exact value from the pages I fetched. Treat #FF500A as a starting point: open wattpad.com, eyedrop the logo or the Sign Up button, and replace the token below if it differs. Everything else derives from this one value.

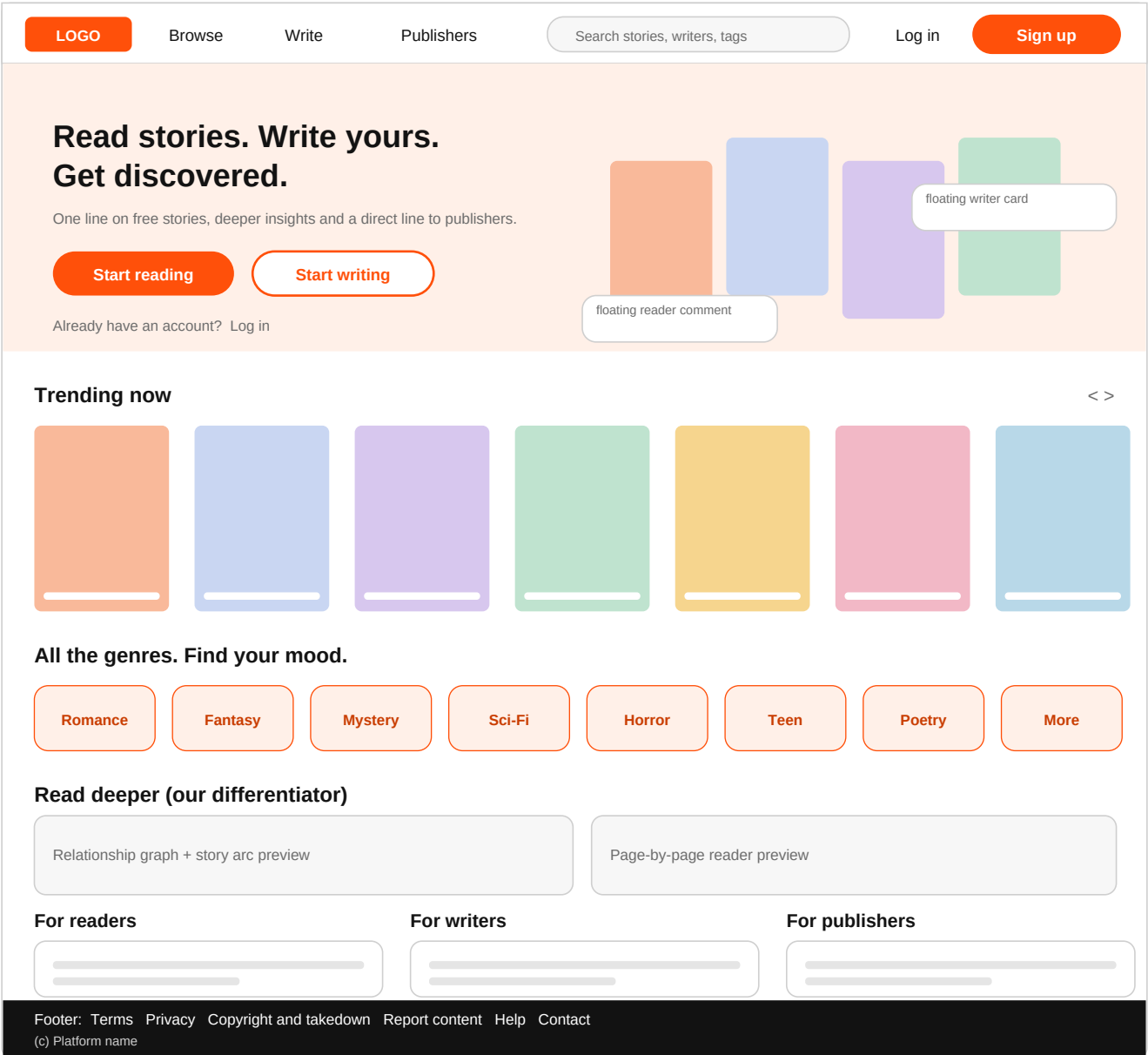
Token	Value	Use
--primary	#FF500A	Primary buttons, links, active tab, progress bars, logo background in wireframes.
--primary-hover	#E04600	Hover and pressed states.
--primary-tint	#FFF0E8	Hero background, selected rows, tag chips, genre tiles.
--ink	#121212	Headings and body text.
--muted	#6B6B6B	Secondary text, captions, meta lines.
--border	#E5E5E5	Cards, inputs, dividers.
--surface	#F7F7F7	Page sections, search box, sidebars.
--success / --danger	#1F9D55 / #D63B2F	Accepted and declined states, errors.
Reader themes	light #FFFFFF / sepia #F4ECD8 / dark #121212 with #E6E6E6 text	Reader settings. Keep body contrast at least 4.5:1.

Aspect	Spec
Typeface	UI: a friendly rounded geometric sans such as Nunito Sans (Google Fonts). Reading text: a book serif such as Lora or Merriweather. (My recommendation; I did not verify which font Wattpad uses.)
Sizes	Hero headline 40/48 desktop, 28/34 mobile. Section titles 24/30. Body 16/24. Meta and captions 13/18. Reader body 18/30 by default.
Buttons	Pill shaped (fully rounded). Primary: solid orange, white text. Secondary: white with orange border and text. Height 44 on mobile, 40 on desktop.
Cards and covers	Covers always 2:3, radius 4 to 8, soft shadow on hover. Catalogue grid: 6 columns desktop, 4 tablet, 3 mobile. Carousels with arrow buttons on desktop and swipe on touch.
Inputs	Radius 8, 1 px border, orange focus ring, 44 px tall.
Chips and tags	Pill chips, tint background, orange text.
Layout	Max content width 1200. Generous white space. Sticky white top bar with a bottom border.
Motion	Light: page turn slide, carousel scroll, hover lifts. Respect prefers-reduced-motion.
Footer	Dark or wave-topped footer with link rows (Terms, Privacy, Copyright and takedown, Report content, Help, Contact).

12.4 Landing page

Section order for guests. Copy below is placeholder text written for you, not Wattpad's wording; change it freely.

#	Section	Content
1	Top bar	Your logo, Browse, Write, For publishers, search box, Log in (text), Sign up (orange pill).
2	Hero	Headline "Read stories. Write yours. Get discovered." Subline "Free stories, deeper insights, and a direct line to publishers." Buttons Start reading and Start writing. Cover collage on the right with one or two floating cards (a short reader comment, a writer stat).
3	Trending now	Horizontal carousel of about 20 covers with title and reads. Arrows on desktop.
4	Genres	Tile grid or carousel of genres with simple illustrated or tinted tiles.
5	Read deeper	Two-column preview: relationship graph and story arc on one side, the page reader on the other. One line each on what readers get.
6	For readers, writers, publishers	Three cards, one per role, each with a one-line promise and a button (Start reading, Publish a book, Apply as publisher).
7	Footer	Terms, Privacy, Copyright and takedown, Report content, Help, Contact, copyright line.



Wireframe: landing page for guests.

12.5 Login and sign-up

- Split layout: a tinted panel with a cover collage on the left, the form on the right. On mobile the collage collapses to a small banner.
- Login: Email, Password, Log in button, Forgot password link, "New here? Sign up" link. Show errors inline, never reveal whether an email exists.
- Sign up: Name, Email, Password (strength hint), a role chooser (Reader, Writer, Apply as publisher). Choosing Publisher reveals company name and website and explains that an admin approves the account.
- A small muted row for "Google sign-in coming soon" is optional; leave the layout space so adding it later does not shift the form.
- Password reset by email link; JWT in httpOnly cookie is preferable to localStorage.

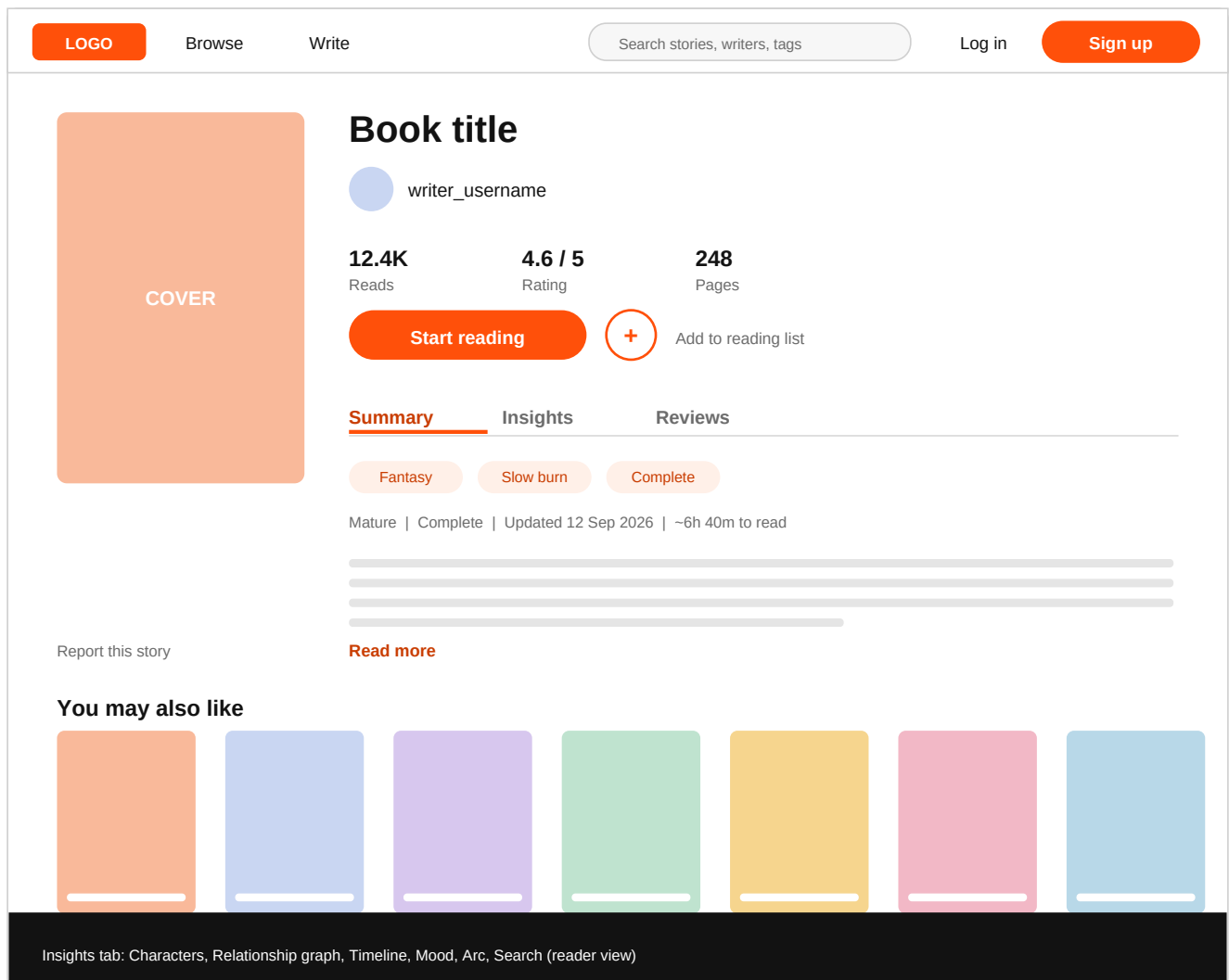
The image shows two side-by-side wireframes for a login and sign-up interface. Both are set against a light orange background with a vertical panel on the left containing three colored squares (orange, blue, purple). The left wireframe is for 'Log in' and features a 'LOGO' button, a 'Sign up' link, input fields for 'Email' and 'Password', a 'Log in' button, and links for 'Forgot password?' and 'New here? Sign up'. The right wireframe is for 'Sign up' and features a 'LOGO' button, a 'Log in' link, a section 'I want to join as' with buttons for 'Reader', 'Writer', and 'Publisher*', input fields for 'Name', 'Email', and 'Password', a 'Create account' button, and a link 'Already have an account? Log in'. Both wireframes include a footer note: 'Google sign-in slot reserved for later'.

Wireframe: login (left) and sign-up (right).

12.6 Book page

This is the Classic template (see section 5). It follows the Wattpad story page: cover left, details right, then summary and a "You may also like" row.

- Stats row: Reads, Rating, Pages. Meta line: maturity, Complete or Ongoing, last updated, estimated reading time.
- Buttons: Start reading (or Continue reading with page number), round + for the reading list, and for publishers a star and "I want to publish".
- Tabs: Summary, Insights, Reviews. Insights shows the reader-visible SceneCraft features; publishers see the Pitch panel here.
- Report this story link and a Content guidelines link at the bottom.
- Guests see a sign-up prompt after the summary.



Wireframe: book page (Classic template).

12.7 Writer, publisher and admin screens

- Writer, publisher and admin use a left sidebar layout with the same orange accent, white cards and Recharts charts (section 7).
- Publisher Discover reuses the catalogue grid and adds filter chips for rating, completion, genre, length and wishlisted.
- Request form is a modal with the fields in section 6; chat is a two-pane layout (conversation list left, thread right) with a header showing the book cover and request status.
- Admin tables use sortable columns, search and inline action buttons (approve, ban, unpublish).

12.8 What not to copy

Use the same structure and colour family, not the same assets. Do not use Wattpad's logo, illustrations, icons, hero images, cover images, headline wording or the name "Wattpad" anywhere in your product, and do not hotlink their images. Layouts, grids and an orange palette are normal design choices; their artwork and branding are not. Draw your own genre illustrations or use licensed or open icon sets, and use seeded sample books with your own placeholder covers.

13. Copyright and moderation

Piece	What to build
Rights checkbox	Required on Publish: "I own this work or have the rights to publish it." Store acceptedTermsAt and termsVersion on the Book. Re-prompt when terms change.
Report button	On books, reviews and writer profiles. Reasons: copyright, plagiarism, abusive content, spam, other. Optional details and, for copyright, the claimant's name and contact.
Admin reports queue	List with status (open, reviewing, closed). Actions: dismiss, unpublish the book, remove a review, issue a strike. Every action is logged with who and when.
Strike policy	Suggested: 3 strikes means suspension. Notify the writer by in-app message and email with the reason and how to appeal.
Copyright and takedown page	Public page explaining how to report infringement, with a contact form and the information a valid notice should include.
Mature content	A Mature flag on books, an 18+ confirmation before reading, and exclusion from default recommendations and the landing page.
Publisher safety	Admin approval before a publisher can send requests; writer can report or block a publisher; messages never include off-platform contact details until the writer shares them.
Data handling	Privacy policy covering stored manuscripts, analytics events and chat. Provide delete-my-account and delete-my-book.
AI processing disclosure	Manuscripts are sent to an LLM provider (OpenRouter) for analysis. State this in the Terms and the publish flow, and confirm the provider's data-retention terms before launch.

A lawyer should review the Terms of Service, takedown procedure and privacy text. The items above give them a complete feature list to review.

14. Rebuild plan on the existing SceneCraft folder

Keep as is

- The BullMQ job graph, workers, OpenRouter services, Redis and Socket.io progress streaming.
- Scene, Character, Relationship, Timeline, Mood, Arc and Continuity models and endpoints.
- React Flow graph, Recharts arc, Framer Motion, Tailwind v4 setup.

Change

- User gets role, status and profile fields. JWT carries the role.
- Document is owned through a Book. Analysis routes move under /books/:id/analysis and gain role and spoiler filtering.
- Pipeline starts on publish (or on demand with a per-writer re-run limit) to control LLM cost: 10 stages per book adds up.
- Frontend routing is split by role with RoleRoute and shared layouts. The notebook workspace becomes the Insights drawer and writer insights page.

Build order

Step	Deliverable	Size
1	Roles and RBAC, admin seed, Book model, image storage service, cover upload, public catalogue and book page (Classic template).	Medium
2	Landing, login and sign-up in the new design system (tokens, nav, buttons, cards, footer).	Medium
3	Upload to pagination to pipeline hook; page-by-page reader with progress, settings and bookmarks.	Large

Step	Deliverable	Size
4	Insights drawer with role gating and spoiler filtering; reading list and My Library.	Medium
5	Reviews and ratings; Book stats; report button and admin reports queue; rights checkbox.	Medium
6	View tracking, DailyStat rollup, writer dashboard including drop-off by page.	Medium
7	Showcase and Notebook templates with live preview.	Small
8	Publisher: apply and approval, Discover, pitch view and pitch card, wishlist.	Medium
9	Publish requests, writer inbox, chat over Socket.io.	Large
10	Admin dashboard and users table; accessibility, empty states, error pages, performance pass.	Medium

Risks to plan for

- **LLM cost and time:** run analysis once at publish, cache everything, cap re-runs, show progress to the writer.
- **Spoilers:** forgetting to filter an endpoint leaks the plot. Route all reader analysis calls through one spoiler service.
- **Fake publishers and spam chat:** admin approval and accepted-only chat address this.
- **Storage and bandwidth:** manuscripts in MongoDB grow quickly and Cloudinary has plan limits on storage and bandwidth; set upload size limits and watch usage.
- **Copyright:** a public platform needs working report and takedown flows from day one.

15. Open questions

Defaults are shown so you can simply say "go with defaults" or change any of them.

#	Question	Default I used
1	Platform name and logo: will you keep "SceneCraft" in the product name or use a new name?	Placeholder "LOGO" and "Platform name".
2	Should chat open only after the writer accepts a request?	Yes.
3	Should readers see every published book for free, or will some be locked later?	All free.
4	Should an admin approve publishers before they can send requests?	Yes.
5	Page size: fixed number of characters per page, or measured to each device screen?	Fixed (about 1,800 characters). Simple and keeps offsets stable.
6	Languages: English only for v1, or multi-language books?	English only, with a language field stored for later.
7	Mature content: allow it with an 18+ gate, or ban it?	Allow with a flag and gate.
8	Can a reader become a writer later from settings?	Yes, one click.
9	Upload limits: maximum file size and page count?	15 MB and about 600 pages for v1.

Sources

Design observations in section 12 come from these pages, read on 3 October 2026. They may change over time.

- Wattpad landing page: <https://www.wattpad.com>
- Wattpad login page: <https://www.wattpad.com/login>
- Wattpad story page (example layout): <https://www.wattpad.com/story/90623050>
- Third-party redesign case study (reading comfort and navigation ideas): https://fueler.io/nirali_makwana/wattpad-website-redesign